Data Engineering Architecture and Tool Selection

From Project Requirements to Defensible Design Decisions

DAEN 459 – Fall 2026

Alexander Abuabara



Learning objectives

- Explain what a data engineering architecture must accomplish.
- Recognize common architectural patterns and their trade-offs.
- Select tools based on requirements, not popularity.
- Design small experiments to validate architecture decisions.
- Produce evidence-based recommendations for a capstone project.

Start with the problem, not the technology

Architecture question

How should data move from its sources to trustworthy, useful outputs under the project's constraints?

- **Stakeholders:** Who needs the data, and for what decisions?
- **Sources:** APIs, files, relational databases, events, sensors.
- **Workloads:** Dashboards, analytics, machine learning, applications.
- **Constraints:** Latency, volume, privacy, budget, skills, operations.
- **Success:** What measurable outcome means the system works?

A reference data engineering architecture

Sources $\longrightarrow$ **Ingestion** $\longrightarrow$ **Storage** $\longrightarrow$ **Transformation** $\longrightarrow$ **Serving**

Cross-cutting capabilities

- Data quality and validation
- Metadata, catalog, lineage
- Security and access controls

Operational capabilities

- Scheduling and orchestration
- Monitoring and alerting
- Testing, deployment, recovery

Architecture is the set of components *and* the contracts between them.

Architectural choices that shape the system

Choice	Option A	Option B
Data movement	Batch: simpler, scheduled	Streaming: lower latency, more operational complexity
Transformation	ETL: transform before loading	ELT: load first, transform in target system
Storage	Warehouse: governed analytical tables	Lake/lakehouse: flexible formats and workloads
Deployment	Managed cloud services	Self-managed or local services
Integration	Point-to-point connections	Reusable pipelines and standard contracts

These are design trade-offs, not universal rankings. Hybrids are common.

Translate needs into nonfunctional requirements

- **Freshness:** Does the output need updates in secs, minutes, or daily?
- **Scale:** Current and expected data sizes, arrival rates, user concurrency.
- **Reliability:** What happens when a source or pipeline fails?
- **Data quality:** What checks define trustworthy data?
- **Governance:** Who can access, retain, or delete data?
- **Maintainability:** Can the capstone team explain, test, operate it?
- **Cost:** Infrastructure charges *plus* development and maintenance.

Key point

Write down measurable targets before comparing tools.

Tool categories and illustrative options

Capability	Example tools	Question to ask
Ingestion	Python, Airbyte, Kafka	Scheduled extraction or event-driven flow?
Storage	PostgreSQL, object storage, DuckDB	Operational queries or analytics?
Transformation	SQL, dbt, Spark, pandas	Data size and transformation complexity?
Orchestration	cron, Prefect, Airflow	How many dependent tasks and retries?
Quality	SQL tests, Great Expectations	What defects must be detected?
Serving	BI dashboard, API, SQL models	Who consumes the outputs and how?

Tools listed are examples, not prescribed stacks.
Check current licensing and service constraints before committing.

A disciplined tool-selection method

1. **Define requirements:** Identify must-haves and nice-to-haves.
2. **Shortlist:** Compare two or three plausible approaches.
3. **Set criteria:** Functionality, integration, learning curve, cost, reliability, maintainability.
4. **Prototype:** Run the same small representative workload.
5. **Measure:** Record latency, throughput, correctness, setup effort, and failure recovery.
6. **Decide:** Document a recommendation and its trade-offs.

Decision principle

Choose the *simplest architecture that meets documented requirements*.

Example: a weighted decision matrix

Criterion	**Weight**	**Option A (1–5)**	**Option B (1–5)**
Requirement fit	0.35	4	5
Team familiarity	0.20	5	2
Operating cost	0.20	4	3
Ease of maintenance	0.15	4	3
Integration effort	0.10	5	3
Weighted score	**1.00**	**4.30**	**3.50**

- Scores are hypothetical examples, not assessments of real products.
- Define each score using evidence from tests or documentation.
- Perform sensitivity analysis: does the decision change if weights change?

Capstone exploration 1: Map your data journey

Team exercise (20–30 minutes)

Draw a one-page diagram from source data to final user-facing output.

- Identify every source, destination, transformation, and dependency.
- Annotate refresh frequency, expected data volume, and data owner.
- Mark where data quality, authentication, and observability belong.
- Distinguish existing components from proposed components.

Deliverable: Initial architecture diagram + three major assumptions.

Capstone exploration 2: Compare two viable designs

Possible contrasts

- Scheduled Python jobs vs. a workflow orchestrator
- Database transformations vs. transformations in Python
- PostgreSQL analytics vs. columnar analytical engine
- File-based ingestion vs. API-driven ingestion

- Use a representative sample and identical success criteria.
- Compare correctness, runtime, complexity, cost, and recoverability.
- Report what would make each architecture preferable.

Deliverable: Two architecture sketches + one-page trade-off analysis.

Capstone exploration 3: Test failure and data quality

- Introduce malformed records, duplicate events, or missing values.
- Simulate an unavailable source or interrupted pipeline run.
- Check whether reruns are safe and outputs remain reproducible.
- Measure how quickly a failure can be detected and diagnosed.
- Document data checks, retry behavior, and recovery procedure.

Guiding question

Can the team demonstrate a system that is trustworthy *when things go wrong*?

Deliverable: Test results + failure/recovery notes.

Suggested mini-project: An architecture decision record

Write a one-page Architecture Decision Record (ADR):

1. Context: what problem must be solved?
2. Decision: which architecture or tool is proposed?
3. Alternatives: what else was considered?
4. Evidence: what did the prototype or benchmark show?
5. Consequences: what complexity, cost, or risk remains?
6. Revisit trigger: what new requirement would change the choice?

A good ADR documents *why* a decision was made, not just which tool was picked.

Capstone checkpoint: What each team should present

1. A concise problem statement and measurable constraints.
2. One current-state and one proposed-state architecture diagram.
3. A shortlist of competing tools or patterns with selection criteria.
4. Results from one targeted proof-of-concept experiment.
5. A decision, its justification, major risks, and next steps.

Discussion prompt

Which architecture decision in your project is most uncertain, and what is the *smallest test* that could reduce that uncertainty?

- Architecture connects requirements, data flow, operations, and outcomes.
- Tools are interchangeable only within the limits of their capabilities and trade-offs.
- Constraints and evidence should drive tool selection.
- Capstone projects offer a chance to test alternatives, not just assemble a stack.
- Document decisions so another team could understand and maintain the system.

Design deliberately.
Test assumptions.
Explain choices.